MA3632 Lecture 9 – Ensemble Methods

Lecture 8 established that a single decision tree is a high-variance estimator: small perturbations to the training set can produce a completely different tree structure, and the instability is a fundamental consequence of the greedy recursive partitioning algorithm. This variance, rather than bias, is often a dominant source of error for sufficiently deep trees. Tree regularisation such as pruning can reduce variance by restricting model complexity, but ensemble methods provide an alternative approach by averaging across multiple trees.

Ensemble methods address this by combining many trees. The key insight, already stated at the end of Lecture 8, is the variance formula for an average of correlated estimators. If $\hat{f}_1, \ldots, \hat{f}_B$ are identically distributed with variance σ^2 and pairwise correlation ρ, then

$$\mathrm{Var}\left(\frac{1}{B}\sum_{b=1}^{B}\hat{f}_b(\mathbf{x})\right) \;=\; \rho\sigma^2 \;+\; \frac{1-\rho}{B}\,\sigma^2. \tag{1}$$

As $B \to \infty$, the second term vanishes and the variance converges to $\rho\sigma^2$. Two consequences follow. First, averaging many trees reduces variance relative to a single tree whenever $\rho < 1$ (with equality only at $\rho = 1$), and the reduction is greater when the trees are less correlated. Second, in the usual setting where trees are positively correlated, reducing ρ lowers the floor $\rho\sigma^2$ that the ensemble variance approaches as B grows – making diversity among trees as important as their number. (In principle ρ could be negative for some ensembles, in which case $\rho\sigma^2$ is not best described as an irreducible floor; this does not arise in the bagging and Random Forest settings considered here, where trees built from resamples of the same data are typically positively correlated.) These two ideas underlie *bagging* and *Random Forests* respectively. A third family, *gradient boosting*, takes a different approach: rather than averaging independently fitted trees, it sequentially adds weak learners to reduce the errors of the current ensemble, often reducing approximation bias.

1 Bagging

Definition 1.1. Let $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ be the training set and let $\mathcal{A}$ be a base learning algorithm. **Bootstrap aggregation** (bagging) proceeds as follows.

(i) For $b = 1, \ldots, B$: draw a bootstrap resample $\mathcal{D}^{(b)} = \{(\mathbf{x}_{i_1}, y_{i_1}), \ldots, (\mathbf{x}_{i_n}, y_{i_n})\}$ by sampling n observations from $\mathcal{D}$ uniformly with replacement. Fit $\hat{f}^{(b)} = \mathcal{A}(\mathcal{D}^{(b)})$.

(ii) For regression, the ensemble prediction is the average: $\hat{f}_{\mathrm{bag}}(\mathbf{x}) = B^{-1}\sum_{b=1}^{B}\hat{f}^{(b)}(\mathbf{x})$.

(iii) For classification, the ensemble prediction is the majority vote: $\hat{f}_{\mathrm{bag}}(\mathbf{x}) = \mathrm{mode}\{\hat{f}^{(b)}(\mathbf{x}) : b = 1, \ldots, B\}$, or equivalently the class with the highest average predicted probability.

Bagging is most effective when the base learner is *unstable*: a small change in the training data produces a large change in the fitted model. Decision trees are the canonical example. Bagging a stable learner (e.g. well-conditioned linear regression, whose fitted coefficients change only slightly under small perturbations to the data) yields little improvement because σ^2 is already small.

Proposition 1.2. *Fix the observed training set $\mathcal{D}$, and let the only remaining randomness be the bootstrap resampling used to generate $\mathcal{D}^{(1)}, \ldots, \mathcal{D}^{(B)}$. Then the conditional expectation of the bagged predictor over this resampling equals the conditional expectation of a single bootstrap predictor:*

$$\mathbb{E}_{\mathrm{boot}}\big[\hat{f}_{\mathrm{bag}}(\mathbf{x}) \mid \mathcal{D}\big] \;=\; \mathbb{E}_{\mathrm{boot}}\big[\hat{f}^{(b)}(\mathbf{x}) \mid \mathcal{D}\big].$$

Proof. By linearity of expectation, $\mathbb{E}_{\text{boot}}[\hat{f}_{\text{bag}} \mid \mathcal{D}] = B^{-1}\sum_{b=1}^{B}\mathbb{E}_{\text{boot}}[\hat{f}^{(b)}(\mathbf{x}) \mid \mathcal{D}]$. Since each $\mathcal{D}^{(b)}$ is drawn identically (given $\mathcal{D}$) as an i.i.d. bootstrap resample, all B terms in the sum are equal, giving the stated result. □

Remark 1.3. Proposition 1.2 shows that, conditional on the observed training set $\mathcal{D}$, averaging B bootstrap predictors changes their variance but not their conditional mean: bootstrap resampling and averaging is a variance-reduction device at this level, not a way of shifting the central tendency of the prediction. This is a more limited statement than saying bagging leaves the overall statistical bias (relative to f^*, and averaged over the original sampling of $\mathcal{D}$ from P) unchanged. In particular, a bootstrap resample contains on average $n(1-(1-1/n)^n) \approx n(1-e^{-1}) \approx 0.632n$ distinct training points, so each tree is trained on a dataset whose distribution differs from that of a tree fitted directly to all n observations of $\mathcal{D}$. Consequently, the bias of the bagged predictor relative to f^* need not be identical to that of the original, unbagged learner; there is no general theorem guaranteeing the direction or magnitude of any such difference, and bagging is primarily motivated by, and justified through, variance reduction rather than a bias guarantee.

1.1 Out-of-bag error estimation

Each bootstrap resample $\mathcal{D}^{(b)}$ omits approximately 37% of the training points – those not selected in any of the n draws. These are called the **out-of-bag** (OOB) observations for tree b.

Proposition 1.4. *The probability that observation i is out-of-bag for a given bootstrap resample is $(1-1/n)^n$, which converges to $e^{-1} \approx 0.368$ as $n \to \infty$.*

Proof. Each of the n draws in the bootstrap resample selects observation i with probability $1/n$, so it is not selected with probability $1-1/n$. Since the n draws are independent, $P(i \text{ is OOB}) = (1-1/n)^n \to e^{-1}$. □

The OOB observations for tree b were not used to fit $\hat{f}^{(b)}$, so they can be used to evaluate it without data leakage. For each training observation i, let $\mathcal{B}_i = \{b : i \notin \mathcal{D}^{(b)}\}$ be the set of trees for which i is OOB. The OOB prediction for observation i is

$$\hat{f}_{\text{OOB}}(\mathbf{x}_i) \;=\; \frac{1}{|\mathcal{B}_i|}\sum_{b\in\mathcal{B}_i}\hat{f}^{(b)}(\mathbf{x}_i),$$

and the **OOB error** is the empirical risk of these predictions over all training observations:

$$\widehat{\text{Err}}_{\text{OOB}} \;=\; \frac{1}{n}\sum_{i=1}^{n}\ell\Big(y_i,\, \hat{f}_{\text{OOB}}(\mathbf{x}_i)\Big)\,.$$

The OOB error provides a convenient estimate of the ensemble's generalisation error and often tracks test error closely in practice. Each prediction $\hat{f}_{\text{OOB}}(\mathbf{x}_i)$ is based on approximately $0.368B$ trees, each trained on a dataset that does not contain i, playing a role analogous to held-out validation rather than being literally equivalent to leave-one-out cross-validation. For large B, the OOB error is a useful practical proxy for the test error without requiring a separate validation set – a significant advantage when data are limited, though its finite-sample bias can depend on the learner and problem at hand.

2 Random Forests

Bagging reduces the variance of a single tree by the factor $(1 + (B-1)\rho)/B$ relative to σ^2 (from equation (1)), but as $B \to \infty$ the residual variance is $\rho\sigma^2$. If the trees are highly correlated (ρ close to 1), bagging provides little benefit in the limit. High correlation arises when a small number of features dominate all trees: regardless of which bootstrap resample is used, the same strong predictor will appear at or near the root of every tree, making their predictions similar.

Random Forests address this by introducing additional randomness at each split.

Definition 2.1. A **Random Forest** is an ensemble of B trees in which each tree is fitted by the following modification of recursive binary splitting: at each node, rather than considering all p features as candidate splits, a random subset of $m \leq p$ features is drawn uniformly without replacement, and the best split is chosen from among these m features only. A new random subset is drawn independently at each node.

The feature subsample size m is the key hyperparameter of a Random Forest. Traditional rules of thumb are $m \approx \lfloor\sqrt{p}\rfloor$ for classification and $m \approx \lfloor p/3\rfloor$ for regression; these have performed well across a wide range of benchmark datasets, though software defaults differ and m can be tuned for a specific problem. Setting $m = p$ recovers bagging.

Random feature subsampling tends to reduce the correlation between trees by preventing the same dominant predictors from being selected at every split: when a dominant feature is excluded from the candidate set at a node, a different feature must be used for that split, producing a different subtree and hence a prediction that is less correlated with those of other trees. The effect is most pronounced at the root, where the dominant feature would otherwise appear in every tree. This reduction in correlation is not guaranteed for every dataset – it depends on how many features carry genuinely useful signal and how it is distributed among them – but it is the principal motivation for the Random Forest modification of bagging.

Remark 2.2. There is a bias–variance trade-off in the choice of m. Smaller m tends to reduce correlation between trees (lower ρ, lower variance of the ensemble) but also means each individual tree is built on less information (higher bias of each tree, slightly higher σ^2). The optimal m balances these two effects; the rule-of-thumb values $\lfloor\sqrt{p}\rfloor$ and $\lfloor p/3\rfloor$ are reasonable starting points but should be tuned by cross-validation or the OOB error for any given dataset.

2.1 Feature importance in Random Forests

The MDI importance of feature j in a single tree (Definition 7.1 in Lecture 8) extends naturally to a Random Forest by averaging over all B trees:

$$\mathrm{MDI}_{\mathrm{RF}}(j) \;=\; \frac{1}{B}\sum_{b=1}^{B}\mathrm{MDI}_b(j),$$

where $\mathrm{MDI}_b(j)$ is the MDI of feature j in tree b. Averaging over many trees stabilises the importance estimates: the high variance of a single tree's MDI (which depends strongly on which training points happened to fall in each node) is reduced by the same mechanism that reduces the variance of the ensemble prediction.

Permutation importance can also be constructed using OOB observations rather than a separate test set. For each tree b and each feature j, one permutes the values of feature j among the OOB observations for tree b and records the increase in that tree's OOB prediction error; the OOB permutation

importance of feature j is the average of this increase across all trees. This is conceptually distinct from simply requesting an OOB score: scikit-learn's `RandomForestClassifier oob_score=True` option computes an OOB accuracy or R^2 score for the fitted ensemble, not a per-feature OOB permutation importance. Per-feature OOB permutation importance of the kind described here is available in third-party packages such as `rfpimp`; scikit-learn's own `permutation_importance` function instead operates on a separate, explicitly supplied evaluation dataset rather than OOB observations.

3 Gradient Boosting

Bagging and Random Forests reduce variance by averaging multiple correlated models, with Random Forests deliberately introducing additional randomness to reduce that correlation between trees. Gradient boosting takes a different approach: rather than averaging independently fitted trees, it sequentially adds weak learners to reduce the errors of the current ensemble, often reducing approximation *bias*.

3.1 The residual-fitting idea

Consider a regression problem with squared loss. Suppose we have a current ensemble prediction $\hat{f}^{(m)}(\mathbf{x})$. The residuals $r_i^{(m)} = y_i - \hat{f}^{(m)}(\mathbf{x}_i)$ represent what the current model has failed to explain. If we fit a new tree $T^{(m+1)}$ to the residuals – treating $r_i^{(m)}$ as the new target – and add it to the ensemble with a small weight $\eta > 0$:

$$\hat{f}^{(m+1)}(\mathbf{x}) \;=\; \hat{f}^{(m)}(\mathbf{x}) \;+\; \eta\, T^{(m+1)}(\mathbf{x}),$$

then the new ensemble reduces the total squared error provided $T^{(m+1)}$ captures some structure in the residuals.

Definition 3.1. Gradient boosting for regression with squared loss proceeds as follows.

(i) Initialise $\hat{f}^{(0)}(\mathbf{x}) = \bar{y} = n^{-1}\sum_{i=1}^{n} y_i$.

(ii) For $m = 1, \dots, M$:

  (a) Compute the pseudo-residuals $r_i^{(m)} = y_i - \hat{f}^{(m-1)}(\mathbf{x}_i)$, $i = 1, \dots, n$.

  (b) Fit a regression tree $T^{(m)}$ to the training set $\{(\mathbf{x}_i, r_i^{(m)})\}_{i=1}^{n}$ with a fixed maximum depth d.

  (c) Update $\hat{f}^{(m)}(\mathbf{x}) = \hat{f}^{(m-1)}(\mathbf{x}) + \eta\, T^{(m)}(\mathbf{x})$.

(iii) Return $\hat{f}^{(M)}$.

The parameters are M (number of trees), $\eta \in (0, 1]$ (learning rate or *shrinkage*), and d (maximum depth of each tree).

Proposition 3.2. *For squared loss $\ell(y, f) = (y - f)^2/2$, the negative gradient of the empirical risk with respect to the current prediction $\hat{f}^{(m-1)}(\mathbf{x}_i)$ is*

$$-\frac{\partial}{\partial \hat{f}}\, \ell(y_i, \hat{f})\bigg|_{\hat{f}=\hat{f}^{(m-1)}(\mathbf{x}_i)} \;=\; y_i - \hat{f}^{(m-1)}(\mathbf{x}_i) \;=\; r_i^{(m)}.$$

That is, the pseudo-residuals are the negative gradient of the loss.

Proof. $\partial \ell(y, \hat{f})/\partial \hat{f} = \hat{f} - y$, so the negative gradient is $y - \hat{f}$, which equals the residual $r_i^{(m)}$ at the current iteration. □

Proposition 3.2 reveals that gradient boosting with squared loss is performing *gradient descent in function space*: at each step, we move the current function $\hat{f}^{(m-1)}$ in the direction of the negative gradient of the empirical risk, with a tree fitted to the gradient as the "step direction" and η as the step size. This interpretation, due to Friedman (2001), extends to arbitrary differentiable loss functions by substituting the appropriate negative gradient for the residuals.

3.2 Generalisation to other loss functions

For binary classification with cross-entropy loss, the negative gradient is not the label residual but the difference between the true label and the current predicted probability: $r_i^{(m)} = y_i - \sigma(\hat{f}^{(m-1)}(\mathbf{x}_i))$, where σ is the sigmoid function. Trees are fitted to these gradient residuals and the ensemble prediction $\hat{f}^{(M)}(\mathbf{x})$ is passed through the sigmoid to produce a class probability. For multiclass problems with softmax loss, one gradient-boosted model is fitted per class.

3.3 Hyperparameters and overfitting

Gradient boosting has three main hyperparameters.
The **number of trees** M controls the total capacity of the ensemble. Unlike Random Forests, where increasing B generally stabilises performance rather than causing the type of overfitting associated with adding more boosting iterations, gradient boosting can overfit as M increases if the learning rate η is not small enough: as M grows, the training loss generally continues to decrease, and generalisation error may eventually increase once the ensemble becomes too complex for the given η and d, though neither the exact point at which this happens nor exact interpolation of the training data is guaranteed in general. Cross-validation or *early stopping* – monitoring the validation error and halting when it starts to rise – is used to select M.
The **learning rate** $\eta \in (0, 1]$ controls how much each tree contributes. Small η requires more trees to achieve the same training error reduction but typically results in better generalisation (more regularisation). There is a well-known trade-off: η and M must be tuned jointly, with small η paired with large M. Setting $\eta \leq 0.1$ and using early stopping is common practice.
The **tree depth** d controls the complexity of each individual tree and hence the types of interactions the model can capture. Depth-1 trees (stumps) represent purely additive main-effect contributions, since each stump splits on only one feature; deeper trees allow interactions between features, with a depth-d tree able to represent interactions involving up to d distinct splitting variables along any single root-to-leaf path. Shallow trees ($d \in \{2, 3, 4\}$) are standard in gradient boosting because the sequential nature of the algorithm allows complex functions to emerge from many simple trees. Deep trees can be used when interactions are important, but increase the risk of overfitting.

Remark 3.3. Gradient boosting also benefits from *stochastic* variants in which each tree is fitted on a random subsample of the training data (analogous to bagging) or a random subsample of features (analogous to Random Forests). These variants reduce overfitting and improve speed. They are implemented in XGBoost (`subsample`, `colsample_bytree`) and LightGBM (`bagging_fraction`, `feature_fraction`).

4 Practical Implementations: XGBoost and LightGBM

The gradient boosting algorithm as stated in Definition 3.1 is the original formulation due to Friedman (2001). Modern implementations improve on it in several ways.

XGBoost (Chen and Guestrin, 2016) uses second-order Taylor expansions of the loss to find better leaf predictions at each step, applies ℓ_1 and ℓ_2 regularisation on the leaf weights, and uses an efficient approximate split-finding algorithm that avoids sorting all features at every node.

LightGBM (Ke et al., 2017) introduces two further algorithmic innovations. Gradient-based One-Side Sampling (GOSS) retains all observations with large gradients (those the current model predicts poorly) but downsamples observations with small gradients, reducing the data volume while preserving the most informative training signal. Exclusive Feature Bundling (EFB) identifies sparse features that rarely take non-zero values simultaneously and bundles them into a single feature, reducing the effective number of features and speeding up split search. These innovations are designed to reduce computational cost, particularly on large and sparse datasets, while maintaining competitive predictive accuracy; the actual speed and accuracy achieved relative to other implementations depend on the dataset, software version, and tuning.

Both implementations expose the same core hyperparameters (M, η, d) and support early stopping, cross-validation, and feature importance. Either can be used through scikit-learn-compatible wrappers.

5 Random Forests vs Gradient Boosting

The two main ensemble families have complementary strengths and weaknesses.

Random Forests are *robust*: they have few hyperparameters to tune (essentially B, m, and the tree depth, with good defaults for all three), increasing B generally stabilises rather than degrades performance, and they are readily parallelisable, because the individual trees can be fitted independently once their bootstrap samples and feature-selection randomness have been generated (their predictions, however, remain statistically correlated, as discussed above). The OOB error provides a convenient estimate of generalisation without a separate validation set. Random Forests tend to work well out of the box and are a sensible first choice.

Gradient boosting *tends to achieve lower error than Random Forests when carefully tuned*, particularly on structured tabular data. However, it requires joint tuning of M, η, and d, is sensitive to overfitting, and is sequential (trees cannot be trained in parallel, since each depends on the residuals of the previous ensemble). It also tends to be less robust to noise and outliers than Random Forests, because each tree is fitted to the residuals of the previous ensemble and large residuals (often caused by outliers) can be amplified.

As a practical guide: start with a Random Forest, assess its performance and feature importances, and move to gradient boosting if further improvement is needed and computational resources allow.

6 Exercises

Exercise 6.1. Let $\hat{f}_1, \ldots, \hat{f}_B$ be identically distributed estimators with $\mathrm{Var}(\hat{f}_b(\mathbf{x})) = \sigma^2$ and $\mathrm{Corr}(\hat{f}_b, \hat{f}_{b'}) = \rho$ for all $b \neq b'$. Derive equation (1):

$$\mathrm{Var}\left(\frac{1}{B}\sum_{b=1}^{B}\hat{f}_b(\mathbf{x})\right) \;=\; \rho\sigma^2 + \frac{1-\rho}{B}\,\sigma^2.$$

Hence show that as $B \to \infty$, the variance converges to $\rho\sigma^2$, and that for $\rho = 0$ (uncorrelated trees), the variance decays to zero at rate $1/B$. Explain why perfectly uncorrelated trees ($\rho = 0$) are unlikely in practice when all trees are fitted on bootstrap resamples of the same training dataset.

Solution. Let $\bar{f} = B^{-1}\sum_{b=1}^{B}\hat{f}_b$. Then

$$\begin{aligned}\mathrm{Var}(\bar{f}) &= \frac{1}{B^2}\mathrm{Var}\left(\sum_{b=1}^{B}\hat{f}_b\right) = \frac{1}{B^2}\left[\sum_{b=1}^{B}\mathrm{Var}(\hat{f}_b) + \sum_{b\neq b'}\mathrm{Cov}(\hat{f}_b, \hat{f}_{b'})\right] \\ &= \frac{1}{B^2}\left[B\sigma^2 + B(B-1)\rho\sigma^2\right] = \frac{\sigma^2}{B} + \frac{(B-1)\rho\sigma^2}{B} \\ &= \rho\sigma^2 + \frac{(1-\rho)\sigma^2}{B},\end{aligned}$$

where the second line uses $\mathrm{Cov}(\hat{f}_b, \hat{f}_{b'}) = \rho\sigma^2$ for all $B(B-1)$ pairs. As $B \to \infty$, $(1-\rho)\sigma^2/B \to 0$, so $\mathrm{Var}(\bar{f}) \to \rho\sigma^2$. For $\rho = 0$, $\mathrm{Var}(\bar{f}) = \sigma^2/B \to 0$.
Zero pairwise correlation ($\rho = 0$) would require the prediction errors of different trees to be linearly uncorrelated – a weaker condition than independence, but still unlikely in practice here, because all bootstrap samples are drawn from the same finite observed dataset $\mathcal{D}$, and the trees tend to exploit the same strong predictive structure present in that dataset: any feature that is genuinely predictive will appear in most trees regardless of the particular bootstrap resample, inducing positive correlation between their predictions. (Note that even independent trees trained on independent samples from the same distribution P would not be a source of the correlation described here; the issue is specifically that all bootstrap resamples are drawn from the one observed, finite $\mathcal{D}$.) Random feature subsampling, as used in Random Forests, can reduce this correlation by preventing the same dominant feature from being used at every split, but does not in general eliminate it: whenever a dominant feature is included in the random subset at a given node, trees will tend to split on it in similar ways. □

Exercise 6.2. Show that the probability of a given observation being out-of-bag for a bootstrap resample of size n is $(1-1/n)^n$, and compute this probability for $n \in \{10, 50, 200, 1000\}$. What is the limit as $n \to \infty$? Hence estimate the number of trees in a Random Forest of $B = 200$ trees for which a given observation is expected to be out-of-bag.

Solution. Each of the n draws in a bootstrap resample selects observation i with probability $1/n$, independently. The probability that observation i is not selected in a single draw is $1 - 1/n$, and the probability it is not selected in any of the n draws is $(1-1/n)^n$.

n	$(1-1/n)^n$	$e^{-1} \approx 0.3679$
10	$(0.9)^{10} \approx 0.3487$	
50	$(0.98)^{50} \approx 0.3642$	
200	$(0.995)^{200} \approx 0.3670$	
1000	$(0.999)^{1000} \approx 0.3677$	

As $n \to \infty$, $(1-1/n)^n \to e^{-1} \approx 0.368$. For $B = 200$ trees and large n, the expected number of trees for which observation i is OOB is $200 \times e^{-1} \approx 73.6$, so roughly 74 trees contribute to the OOB prediction for each training observation. □

Exercise 6.3. A gradient boosting model with squared loss is initialised at $\hat{f}^{(0)}(\mathbf{x}) = \bar{y}$ and trained for two steps with learning rate $\eta = 0.5$. The training set consists of four observations:

i	x_i	y_i	$\hat{f}^{(0)}(x_i)$
1	1	3	2.5
2	2	4	2.5
3	3	1	2.5
4	4	2	2.5

At step $m = 1$, a depth-1 regression tree $T^{(1)}$ is fitted to the residuals $r_i^{(1)} = y_i - 2.5$ and produces the following predictions: $T^{(1)}(x) = 1.0$ for $x \leq 2.5$ and $T^{(1)}(x) = -1.0$ for $x > 2.5$.
Compute the residuals $r_i^{(1)}$ and the updated predictions $\hat{f}^{(1)}(x_i)$. Then compute the second-step residuals $r_i^{(2)}$ and state what a depth-1 tree fitted to them would predict (you do not need to find the optimal split threshold; describe qualitatively what pattern the residuals show). Compute the training MSE after each step and confirm it decreases.

Solution. **Step 1.** Residuals: $r_i^{(1)} = y_i - 2.5$, giving $r^{(1)} = (0.5, 1.5, -1.5, -0.5)$.
Updated predictions: $\hat{f}^{(1)}(x_i) = 2.5 + 0.5 \times T^{(1)}(x_i)$. For $i = 1, 2$ (where $x \leq 2.5$): $\hat{f}^{(1)} = 2.5 + 0.5(1.0) = 3.0$. For $i = 3, 4$ (where $x > 2.5$): $\hat{f}^{(1)} = 2.5 + 0.5(-1.0) = 2.0$.
Training MSE after step 1: $(1/4)[(3-3)^2 + (4-3)^2 + (1-2)^2 + (2-2)^2] = (1/4)(0+1+1+0) = 0.5$.
Compare to initial MSE: $(1/4)[(3-2.5)^2 + (4-2.5)^2 + (1-2.5)^2 + (2-2.5)^2] = (1/4)(0.25 + 2.25 + 2.25 + 0.25) = 1.25$. The MSE has decreased from 1.25 to 0.5.
Step 2. Residuals: $r_i^{(2)} = y_i - \hat{f}^{(1)}(x_i)$, giving $(3-3, 4-3, 1-2, 2-2) = (0, 1, -1, 0)$. The residuals are now larger for observations 2 and 3 (which were only partially corrected) and zero for observations 1 and 4 (which were exactly fitted). A depth-1 regression tree fitted to these residuals and splitting at $x = 2.5$ predicts the mean residual in each leaf: the left leaf ($x \leq 2.5$) contains residuals $(0, 1)$, with mean 0.5, and the right leaf ($x > 2.5$) contains residuals $(-1, 0)$, with mean -0.5. Hence, with $\eta = 0.5$, the updated predictions are

$$\hat{f}^{(2)}(x_i) = \hat{f}^{(1)}(x_i) + 0.5 \times T^{(2)}(x_i) = (3.0+0.25,\ 3.0+0.25,\ 2.0-0.25,\ 2.0-0.25) = (3.25, 3.25, 1.75, 1.75).$$

The training MSE after step 2 is

$$\frac{1}{4}\left[(3-3.25)^2 + (4-3.25)^2 + (1-1.75)^2 + (2-1.75)^2\right] = \frac{1}{4}(0.0625+0.5625+0.5625+0.0625) = 0.3125.$$

Thus the training MSE decreases from 1.25 initially, to 0.5 after step 1, to 0.3125 after step 2, confirming the monotone decrease requested. □

Exercise 6.4. A Random Forest is trained with $B = 100$ trees on a dataset with $p = 20$ features. The default feature subsample size $m = \lfloor \sqrt{20} \rfloor = 4$ is used.
Suppose an oracle tells you that the pairwise correlation between individual tree predictions (under bagging, i.e. $m = 20$) is $\rho_{\text{bag}} = 0.6$ and the per-tree variance is $\sigma^2 = 0.04$. After switching to $m = 4$, the correlation drops to $\rho_{\text{RF}} = 0.2$ and the per-tree variance increases slightly to $\sigma^2_{\text{RF}} = 0.05$ (because individual trees are weaker).
Compute the ensemble variance under bagging and under Random Forests, both for $B = 100$ and in the limit $B \to \infty$. Which method gives lower variance in the limit? By what factor does the Random Forest reduce the limiting variance relative to bagging?

Solution. Using $\text{Var}(\bar{f}) = \rho\sigma^2 + (1-\rho)\sigma^2/B$:
Bagging ($\rho = 0.6$, $\sigma^2 = 0.04$):

$$B = 100: \quad 0.6(0.04) + (0.4)(0.04)/100 = 0.024 + 0.00016 = 0.02416.$$
$$B \to \infty: \quad \rho\sigma^2 = 0.6 \times 0.04 = 0.024.$$

Random Forest ($\rho = 0.2$, $\sigma^2 = 0.05$):

$$B = 100: \quad 0.2(0.05) + (0.8)(0.05)/100 = 0.010 + 0.0004 = 0.0104.$$
$$B \to \infty: \quad \rho\sigma^2 = 0.2 \times 0.05 = 0.010.$$

In the limit, the Random Forest variance (0.010) is lower than the bagging variance (0.024) by a factor of $0.024/0.010 = 2.4$. Despite the slight increase in per-tree variance (σ^2 rising from 0.04 to 0.05), the substantial reduction in correlation (ρ falling from 0.6 to 0.2) more than compensates, giving a net reduction in ensemble variance of approximately 58.3% ($1 - 0.010/0.024 \approx 0.583$). This illustrates how a sufficiently large reduction in correlation can more than compensate for an increase in individual-tree variance. □

Further Reading

Breiman, L. (1996). Bagging predictors. *Machine Learning*, 24(2), 123–140. The original paper introducing bootstrap aggregation and the variance-reduction argument used in Section 1.

Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32. The foundational paper defining Random Forests and introducing feature subsampling at each split; contains the original variance analysis and OOB error discussion used in Sections 1 and 2.

Friedman, J. H. (2001). Greedy function approximation: a gradient boosting machine. *Annals of Statistics*, 29(5), 1189–1232. The paper that introduced the functional gradient descent interpretation of boosting used in Section 3.

Chen, T. and Guestrin, C. (2016). XGBoost: a scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794. Introduces the second-order Taylor expansion of the loss and the regularised objective underlying the XGBoost implementation discussed in Section 4.

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., and Liu, T.-Y. (2017). LightGBM: a highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems*, 30, 3146–3154. Introduces GOSS and EFB, the two algorithmic innovations discussed in Section 4.

Lindholm, A., Wahlström, N., Lindsten, F., and Schön, T. B. (2022). *Machine Learning: A First Course for Engineers and Scientists*. Cambridge University Press. Treats bagging, Random Forests, and boosting as a unified family of ensemble methods, at a level consistent with this lecture.

James, G., Witten, D., Hastie, T., Tibshirani, R., and Taylor, J. (2023). *An Introduction to Statistical Learning with Applications in Python*. Springer. Chapter 8 covers bagging, Random Forests, and boosting in detail, complementing the treatment here.

Hastie, T., Tibshirani, R., and Friedman, J. (2009). *The Elements of Statistical Learning* (2nd ed.), Chapters 10 and 15. Springer. A standard graduate-level reference for boosting and Random Forests, covering the theoretical properties of both methods in depth.

References

Breiman, L., 1996. Bagging predictors. *Machine Learning, 24*(2), pp.123–140.

Breiman, L., 2001. Random forests. *Machine Learning, 45*(1), pp.5–32.

Chen, T. and Guestrin, C., 2016, August. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794).

Friedman, J.H., 2001. Greedy function approximation: a gradient boosting machine. *Annals of Statistics, 29*(5), pp.1189–1232.

Hastie, T., Tibshirani, R. and Friedman, J., 2009. *The Elements of Statistical Learning* (2nd ed.). Springer.

James, G., Witten, D., Hastie, T., Tibshirani, R. and Taylor, J., 2023. *An Introduction to Statistical Learning with Applications in Python*. Springer.

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q. and Liu, T.Y., 2017. LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems, 30*.

Lindholm, A., Wahlström, N., Lindsten, F. and Schön, T.B., 2022. *Machine Learning: A First Course for Engineers and Scientists*. Cambridge University Press.